UML Diagrams in System Design

Advanced Study Notes: Visualizing Architecture & Object Interactions

1. Why UML? The Language of Architecture

In software engineering, describing a complex system through text is highly inefficient. Unified Modeling Language (UML) bridges the gap between a developer's mental model and actual implementation. It allows engineers to visually communicate component structures, relationships, and data flows.

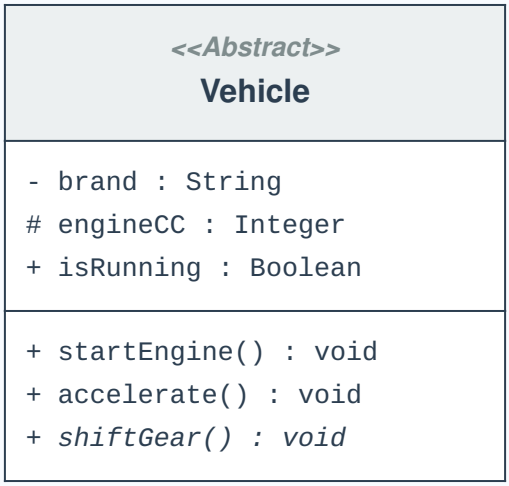
While UML has 14 official diagram types divided into **Structural** (Static) and **Behavioral** (Dynamic) categories, Low-Level Design (LLD) interviews and industry standard documentation heavily prioritize just two: **Class Diagrams** and **Sequence Diagrams**.

2. Deep Dive: Class Diagrams (Structural)

Class diagrams are the backbone of Object-Oriented design. They define the "blueprint" of the application, showing exactly what entities exist and what data they hold.

The Anatomy of a UML Class

A class is drawn as a partitioned rectangle. Understanding the notation is critical for precise communication.



Access Modifier Cheat Sheet

- \+ **Public:** Accessible from anywhere.

- \- **Private:** Strictly accessible only within the defining class. (Crucial for Encapsulation/Data Hiding).
- # **Protected:** Accessible within the class and its subclasses (child classes).
- *Italics:* If a method is italicized, it denotes an abstract (virtual) method that must be implemented by a child class.

3. Object Relationships: The UML Connectors

Classes rarely operate in a vacuum. How they connect defines the robustness of your system. Here are the core relationship connectors used in industry.

Type	Relationship	Visual Connector	Definition & Real-World Example
Inheritance (Generalization)	**IS-A**		A child extends a parent. *Example: ElectricCar IS-A Car.*
Simple Association	**HAS-A** (Loose)		A simple, independent link between objects. *Example: User HAS-A BankAccount.*
Aggregation	**HAS-A** (Container)		A "part-of" relationship where parts can exist independently of the whole. *Example: Room HAS-A Chair. (If the room is destroyed, the chair still exists).*
Composition	**HAS-A** (Strict)		A strict "part-of" relationship. The part cannot exist without the whole. *Example: Chair HAS-A ChairLeg. (A chair leg has no purpose without the chair).*

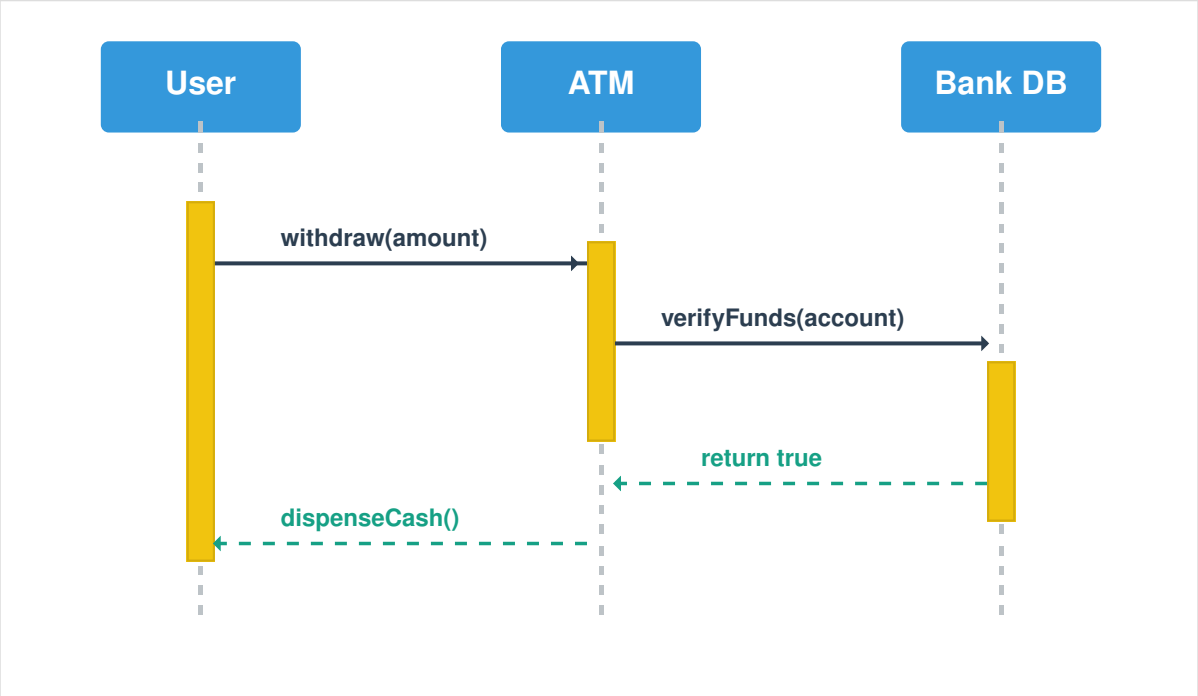
> 💡 **Pro-Tip: Ambiguity in System Design**
> In interviews, the line between Aggregation and Composition can be blurry. For example, does a `Menu` exist without a `Restaurant`? It depends purely on how you design the business logic. State your assumptions clearly to the interviewer (e.g., "I am using Composition here because deleting the Restaurant should cascade and delete its specific Menu in our database.").

4. Sequence Diagrams (Behavioral)

While Class Diagrams show the skeleton, Sequence Diagrams show the nervous system. They map exactly how objects pass messages to fulfill a specific Use Case (like an ATM withdrawal).

Visualizing a Sequence Flow

Below is a visual representation of how Lifelines, Activation Bars, and Messages interact over time (top to bottom).



Key Terminology

- **Lifeline (Dashed Vertical Line):** Represents the lifespan of the object during the interaction.
- **Activation Bar (Yellow Rectangle):** Indicates the exact period when an object is actively executing logic or processing a request.
- **Synchronous Message (Solid Line + Filled Arrow):** The sender waits for a response before doing anything else (e.g., verifying a pin).

- **Asynchronous Message (Solid Line + Open Arrow):** The sender fires the message and immediately continues its work (e.g., triggering a background notification).
- **Return Message (Dashed Line):** The result sent back from a synchronous call.

> 💡 **Pro-Tip: Managing Complexity**
> Do not try to fit every edge case into one sequence diagram. Sequence diagrams are meant to model a *single flow*. You should have one diagram for the "Happy Path" (Successful ATM Withdrawal) and separate diagrams or **`Alt` fragments** (If-Else logic blocks) for edge cases like "Insufficient Funds".